Part 3: Cross-Language Comparison

Course: 23CSE455 Design Patterns
Team: B14
Case study: Design Pattern Identification and Evolution in a Next.js-Based Digital News and Publishing Application
Working scope: Two confirmed patterns, implemented in Java and C++

1. Objective and scope

Part 3 compares equivalent, small implementations of two patterns confirmed in Part 1: **Strategy — LLM Provider Streaming Strategy** and **Observer — Auth State Observer**. RBAC is intentionally excluded to follow the team's requested two-pattern scope. The examples are deterministic teaching/test snippets; they are not complete ports of the Next.js application and do not call real LLM services or Supabase.

The assignment instructions specify Java, Python, JavaScript, and C++. This working bundle implements Java and C++ only. Unless the instructor approved a reduced language scope, Python and JavaScript remain outstanding for final compliance.

2. Pattern selection and source evidence

2.1 Strategy

Part 1 identifies `LLM Provider Streaming Strategy` in `code/features/ai-assistant/llmService.ts`. The provider-specific functions are `streamClaude`, `streamOpenAI`, and `streamGemini`; `sendMessage` selects a provider using `params.provider`. The baseline has an implicit function contract rather than a declared strategy interface, and the selector is a hard-coded conditional chain.

The Java and C++ examples use explicit strategy interfaces/abstract classes so their roles can be compared clearly. This is a structural teaching adaptation, not a claim that the original TypeScript code declares the same interface.

2.2 Observer

Part 1 identifies `Auth State Observer` in `AuthProvider.tsx`, `AuthContext.tsx`, and `useSession.ts`. The original subject is `supabase.auth` from the Supabase dependency; `AuthProvider` supplies the callback to `onAuthStateChange`, updates auth/session state, and unsubscribes during cleanup. No separate subject class is invented for the original application.

The Java and C++ examples demonstrate subscribers receiving auth-state events and the effect of unsubscribing. They do not connect to Supabase.

3. Relationship to completed Part 2 evidence

The uploaded Part 2 records document three changes:

- **CH01 — provider inference:** adds `inferProviderFromModel()` and updates `generateBlogPost()` to use it. Its implementation note explicitly says not every provider-selection branch was refactored. This is relevant to the Strategy area but does not alone prove the original strategy collaboration was fully redesigned.
- **CH02 — publication validation:** extracts title and article-content checks into named helper functions. The record claims no new GoF pattern.
- **CH03 — generated-post JSON parsing:** extracts shared parsing/error handling into `parseGeneratedPostJson()`. This is a duplication refactor, not a new pattern claim for this comparison.

Source-snapshot caveat: In the uploaded archive, the `B14/code/features/ai-assistant/llmService.ts` copy still contains the original provider-selection branches and does not contain the helper names described in CH01/CH03. The Part 2 implementation notes and test evidence are included, but the source changes appear not to have been merged into that copied `code/` snapshot. Accordingly, this Part 3 uses the verified Part 1 baseline behaviour and documents the Part 2 evidence without claiming to benchmark the post-refactoring source. Update this note and re-check the source if the refactored code is merged.

4. Common test specification

`test-cases/common_test_cases.csv` is the language-independent source of test inputs and expected results.

- Strategy: S01 Claude selection; S02 OpenAI selection; S03 Gemini selection; S04 unsupported provider rejection.
- Observer: O01 sign-in notification; O02 sign-out notification; O03 no notification after unsubscribe; O04 notification delivered to multiple active observers.

Each Java and C++ test runner encodes the same expected behaviour. The recorded results are:

Pattern	Java	C++
Strategy	4/4 passed	4/4 passed
Observer	4/4 passed	4/4 passed

Actual compilation and test output is in `evidence/test_results.txt`.

5. Measurement method

- **LOC:** production files only; blank lines, comments, preprocessor guards and tests excluded.
- **Cyclomatic complexity:** function-by-function manual McCabe audit using `M = 1 + D`. Each `if` and loop counts as one decision; each non-default `switch` case counts as one; abstract declarations and library internals are excluded. The function-level decisions and arithmetic are documented in `evidence/complexity_audit.csv` and `evidence/complexity_audit.md`. This is a transparent source-level audit, not output from an external static-analysis package.
- **Timing:** compilation occurs before measurement; one warm-up run is followed by five measured process launches on the same environment. `median_process_ms` includes process startup and the test harness and excludes compilation.
- **Environment:** see `evidence/environment.txt`.
- **Raw timing:** see `evidence/timing_raw.csv`.

6. Results

Pattern	Language	Production LOC	Max function CC (manual McCabe audit)	Median process time (ms)
Strategy	Java	33	4	36.116
Observer	Java	24	2	42.995
Strategy	C++	35	4	1.115
Observer	C++	30	2	1.070

The process times are specific to this machine. Because the examples are very small, JVM startup/runtime initialization affects Java timings substantially. These measurements should not be interpreted as a universal language-performance ranking. LOC and complexity are also not sufficient on their own to determine which design is better.

7. Language comparison

- Java uses an interface for the strategy contract and concrete strategy classes; C++ uses an abstract base class with virtual dispatch.
- Both implementations select one provider strategy and return deterministic chunks matching the shared fixtures.
- Both Observer implementations keep a collection of listeners, publish events to active listeners, and support unsubscribe.
- Java manages listener references through object references and standard collection utilities. C++ uses non-owning pointers in this small example;

production C++ code would need a carefully defined lifetime/ownership policy.
- The original Observer uses Supabase's subscription API and React Context; the small examples isolate the event/observer behaviour for cross-language comparison.

8. Files and reproducibility

- `strategy/java/` and `strategy/cpp/`: Strategy implementations and tests.
- `observer/java/` and `observer/cpp/`: Observer implementations and tests.
- `test-cases/common_test_cases.csv`: shared test specification.
- `cross_language.csv`: one row per selected pattern/language implementation.
- `evidence/test_results.txt`: compile and test output.
- `evidence/timing_raw.csv`: five measured runs per implementation.
- `evidence/complexity_audit.csv` and `evidence/complexity_audit.md`: function-by-function cyclomatic complexity audit.
- `evidence/environment.txt`: toolchain versions.
- `llm/part3_prompt.txt`: reusable prompts.
- `llm/llm.csv`: header-only template; actual AI interactions must be recorded, not fabricated.

9. Limitations and remaining work

1. Only Java and C++ are implemented in this working bundle; Python and JavaScript are required by the written course instructions unless the instructor approved otherwise.
2. Complexity values are now documented in a function-level manual audit. If the course requires a specific static-analysis tool, run it and reconcile any counting-convention differences; this environment could not install an external analyzer because package network access was unavailable.
3. The code snippets are representative demonstrations, not ports of the original production application.
4. The copied source tree does not include the Part 2 helper changes described in the implementation notes; reconcile the source snapshot before claiming post-refactoring comparison.
5. AI usage records must be completed from actual interactions and verified outputs.

10. Conclusion

The two selected patterns can be expressed and tested in both Java and C++ while preserving the specified behaviours. The common fixtures and raw evidence make the preliminary comparison reproducible. The work is a two-pattern,

two-language Part 3 draft; it should not be represented as a completed four-language submission until the remaining language implementations and verification requirements are addressed.